7.3.1. Guideline: use std::memory_order_seq_cst for prototyping
7.3.2. Guideline: use a lock-free memory reclamation scheme
7.3.3. Guideline: watch out for the ABA problem
7.3.4. Guideline: identify busy-wait loops and help the other thread
7.4. Summary
Chapter 8. Designing concurrent code
8.1. Techniques for dividing work between threads
8.1.1. Dividing data between threads before processing begins
8.1.2. Dividing data recursively
8.1.3. Dividing work by task type
8.2. Factors affecting the performance of concurrent code
8.2.1. How many processors?
8.2.2. Data contention and cache ping-pong
8.2.3. False sharing
8.2.4. How close is your data?
8.2.5. Oversubscription and excessive task switching
8.3. Designing data structures for multithreaded performance
8.3.1. Dividing array elements for complex operations
8.3.2. Data access patterns in other data structures
8.4. Additional considerations when designing for concurrency
8.4.1. Exception safety in parallel algorithms
8.4.2. Scalability and Amdahl’s law
8.4.3. Hiding latency with multiple threads